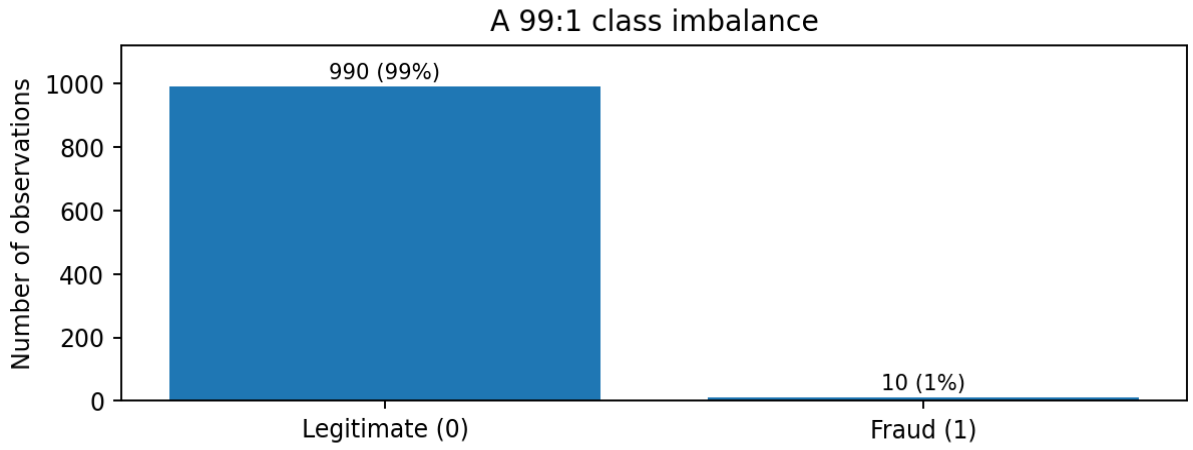
IMBALANCED DATASETS

Interview Revision Guide

A practical, visual guide to metrics, sampling, ensembles, class weighting, cost-sensitive learning and threshold tuning.



Central idea: class imbalance is not solved by automatically forcing a 50:50 dataset. The goal is to build a model that performs well on the class and errors that matter, using leakage-safe evaluation.

How to use this guide

Read the fundamentals first, then focus on the intervention you are least comfortable with. Every technique is explained through its mechanism, a small example, trade-offs, and interview-ready takeaways.

Scope: concise recap of undersampling/oversampling; deeper treatment of hybrid sampling, ensembles, class weighting, cost-sensitive learning, threshold tuning and practical evaluation.

1. What is an imbalanced dataset?

A classification dataset is imbalanced when class frequencies differ substantially. In binary classification, one class is called the majority class and the other the minority class. In multiclass problems, there may be several classes with very different frequencies.

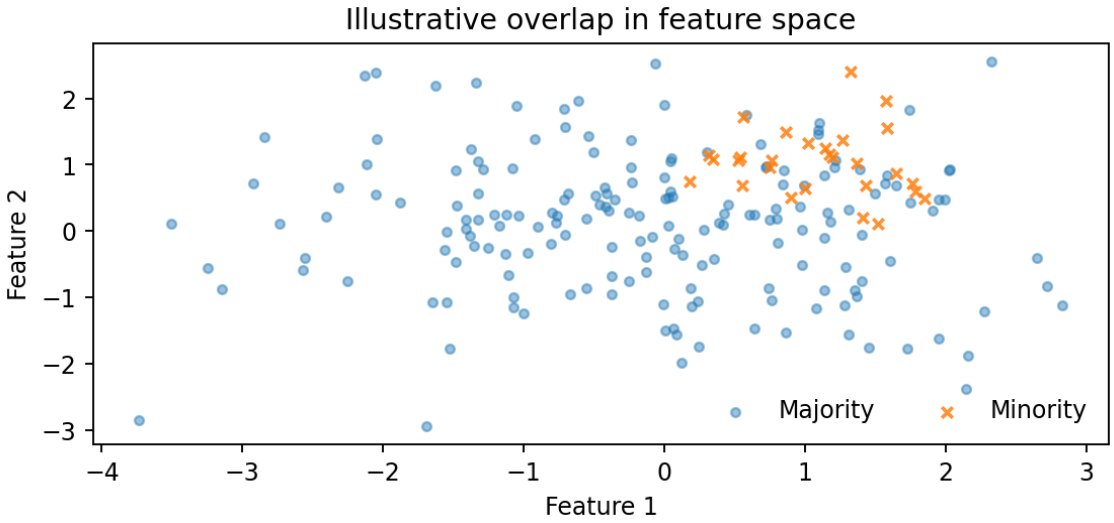
Example	Majority	Minority	Imbalance ratio
Fraud	9,900 legitimate	100 fraud	99:1
Medical screening	950 no disease	50 disease	19:1
Machine failure	9,700 normal	300 failure	32.3:1

Imbalance ratio

A common convention is majority count divided by minority count. If there are 990 negatives and 10 positives, the ratio is 990/10 = 99:1. State the convention when reporting it, especially in multiclass problems.

Why the ratio alone is not enough

- A 99:1 dataset can still be learnable if classes are well separated and enough positive examples exist.
- A less imbalanced dataset can be difficult when classes overlap, labels are noisy, or the minority class is heterogeneous.
- The business cost of errors matters: missing a cancer case is not equivalent to sending one extra screening alert.



The scatter plot is illustrative: overlap and geometry affect difficulty independently of class counts.

Interview answer: Imbalance is a distribution property, not by itself proof that a model is bad. Diagnose it together with class overlap, data quality, sample size and the error objective.

2. Why accuracy can mislead

Suppose 1,000 transactions contain 990 legitimate transactions and 10 fraudulent transactions. A model predicts every transaction as legitimate.

	Predicted fraud	Predicted legitimate
Actual fraud	TP = 0	FN = 10
Actual legitimate	FP = 0	TN = 990

Accuracy = (TP + TN) / N = 990 / 1,000 = 99%. Yet fraud recall is 0%: every fraud was missed.

Core metrics

Metric	Formula	Interpretation
Precision	TP / (TP + FP)	Of predicted positives, how many are positive?
Recall / sensitivity	TP / (TP + FN)	Of actual positives, how many were found?
Specificity	TN / (TN + FP)	Of actual negatives, how many were correctly rejected?
F1-score	2PR / (P + R)	Harmonic mean of precision and recall
Balanced accuracy	(Recall + Specificity) / 2	Average class-wise recall in binary classification

ROC-AUC vs PR-AUC

- ROC-AUC summarizes ranking across true-positive and false-positive rates; it can look strong when negatives vastly outnumber positives.
- Precision–recall curves focus on positive-class retrieval and are often more informative when positives are rare.
- Average precision is a common scalar summary of a precision–recall curve; it is not exactly the same as every possible numerical integration of PR-AUC.

Choose metrics from the objective. High cost of missed positives often prioritizes recall; limited investigation capacity may prioritize precision; F1 balances precision and recall but does not encode business costs.

```
Python: from sklearn.metrics import confusion_matrix, classification_report, average_precision_score, roc_auc_score
```

3. A map of the main strategies

There are several intervention points. They solve related but different problems.

Sampling → Changes training examples

Class weighting → Changes training loss

Ensembles → Changes learning procedure

Threshold tuning → Changes final decision rule

Family	What changes?	Examples
Data-level sampling	Training examples or their counts	Random under/oversampling, SMOTE
Hybrid sampling	Minority generation plus cleaning	SMOTE-Tomek, SMOTE-ENN
Ensemble methods	How learners are trained and combined	Balanced Random Forest, EasyEnsemble
Class weighting	Contribution of errors to training loss	class_weight='balanced'
Cost-sensitive learning	Objective or decisions account for error costs	Weighted loss, expected-cost minimization
Threshold tuning	Probability-to-label decision rule	Choose threshold on validation data

Undersampling recap

Reduces the majority class. It can save computation and remove redundant examples, but may discard useful information. Cleaning methods such as Tomek Links and ENN focus on local class overlap rather than random deletion.

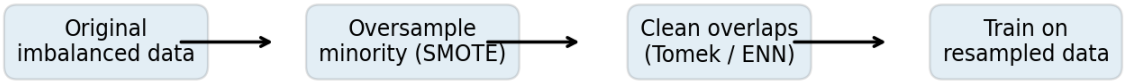
Oversampling recap

Increases minority representation. Random oversampling duplicates observations; SMOTE interpolates between minority neighbors. Synthetic samples can help, but may reinforce noise or create unrealistic points in overlapping regions.

Do not assume 50:50 is always best. Treat sampling ratio as a hyperparameter and choose it using validation performance aligned with the task.

4. Hybrid sampling: oversample, then clean

Hybrid methods combine minority-class oversampling with an undersampling/cleaning step. The intuition is to improve minority representation and then remove selected observations in confusing regions.



SMOTE-Tomek

- Stage 1: SMOTE creates synthetic minority observations using minority neighbors.
- Stage 2: Tomek Links identifies mutual nearest-neighbor pairs across classes.
- The default SMOTETomek cleaning behavior uses Tomek Links with its default strategy; implementations/parameters can be configured. Do not assume both members of every pair are removed.

SMOTE-ENN

- Stage 1: SMOTE increases minority representation.
- Stage 2: ENN removes targeted observations whose neighboring labels fail the selected consistency rule.
- ENN often performs broader cleaning than Tomek Links because it evaluates neighborhood label consistency, not just mutual cross-class pairs.

Question	SMOTE-Tomek	SMOTE-ENN
Cleaning rule	Mutual cross-class nearest neighbors	Neighbor label consistency
Typical cleaning	More targeted	Often more aggressive
Main risk	Useful boundary observations may be removed	More information can be discarded

Code

```
from imblearn.combine import SMOTETomek, SMOTEENN
from imblearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

sampler = SMOTETomek(random_state=42)
# Alternative: sampler = SMOTEENN(random_state=42)

model = Pipeline([
    ("scale", StandardScaler()),
    ("sample", sampler),
    ("clf", LogisticRegression(max_iter=1000))
])
model.fit(X_train, y_train)
```

Fit samplers only inside the training pipeline. StandardScaler is fitted within the pipeline too, preventing statistics from leaking from validation/test data.

5. Imbalance-aware ensembles: the intuition

An ensemble combines multiple learners. Imbalance-aware ensembles alter how the learners see the training data so that majority examples do not dominate every learner in the same way.

Balanced Random Forest

- Builds a forest of trees using balanced samples: each tree trains on a balanced bootstrap/subsample strategy rather than letting the majority class dominate every tree.
- Different trees can see different majority examples, preserving more majority information across the ensemble than one aggressively undersampled dataset might.
- Useful as a strong baseline when majority-class redundancy is high.

EasyEnsemble

- Creates multiple balanced subsets by repeatedly under-sampling the majority class.
- Trains an ensemble learner on each subset and combines their outputs.
- Different subsets let the ensemble learn from different portions of the majority class.

RUSBoost

- Combines random under-sampling with AdaBoost-style sequential learning.
- Each boosting round trains a weak learner on a resampled dataset; boosting updates learner importance based on performance.
- Useful when you want to combine resampling with boosting, but it can still lose information through under-sampling.

Conceptual diagram

Approach	What is diversified?
Balanced Random Forest	Balanced samples across trees
EasyEnsemble	Majority subsets and their trained ensembles
RUSBoost	Sequential boosted learners with random under-sampling

These are not identical algorithms. For interviews, describe the sampling strategy and how predictions are combined rather than calling all three simply 'balanced random forests'.

7. Ensemble methods: let multiple models learn from balanced samples

We have 1,000 transactions: 990 legitimate (Class 0) and 10 fraud (Class 1). A standard learner sees far more legitimate examples. An imbalance-aware ensemble changes how its models are trained so the minority class is represented in each learner's training sample.

Step 1 — Build different balanced subsets

Training subset	Legitimate examples	Fraud examples	What changes?
Tree 1	10 selected from the 990	The 10 fraud examples	Learns from subset A
Tree 2	A different 10 selected	The 10 fraud examples	Learns from subset B
Tree 3	Another 10 selected	The 10 fraud examples	Learns from subset C

These are deliberately tiny teaching subsets. Real implementations have their own sampling and bootstrap rules; the point is that different learners see different selections of majority examples.

Step 2 — Train learners and combine predictions

New transaction	Tree 1	Tree 2	Tree 3	Simple majority vote
Prediction	Fraud	Fraud	Legitimate	Fraud

Why can this help? Every learner gets a more balanced training view. Using different majority subsets lets the ensemble learn from more of the majority data across the full collection of learners than one single undersampled dataset would.

Three interview names

Method	Core mechanism	Trade-off
Balanced Random Forest	Trees train on balanced class samples.	Can discard majority information per tree; ensemble may recover diversity across trees.
EasyEnsemble	Train multiple learners on different majority undersamples, then combine them.	More training work, but sees multiple majority subsets.
RUSBoost	Combines random undersampling with sequential boosting.	Can focus learners over rounds, but undersampling can still discard useful examples.

Keep this distinction: an ensemble changes the training procedure and combines learners. It is not the same as class weighting, which changes the contribution of errors to a training objective.

Mini-check: If every tree used exactly the same small majority subset, what benefit would different subsets provide? Answer: less diversity in which majority examples the ensemble learns from.

8. Class weighting: make some training errors count more

Keep all 1,000 transactions: 990 legitimate and 10 fraud. Instead of removing rows or creating copies, we tell the learning algorithm that errors on fraud examples should contribute more strongly to its training objective.

Step 1 — Imagine two equally sized errors

Error	Class weight	Illustrative weighted contribution
One legitimate example is misclassified	1	1 × loss = 1 unit
One fraud example is misclassified	10	10 × loss = 10 units

The numbers 1 and 10 are chosen only to explain the idea. A real weight depends on the chosen policy or estimator setting.

Step 2 — What changes during training?

The model adjusts its parameters to reduce the total loss. If a fraud error receives a larger weight, reducing that error becomes more important to the optimizer. The data rows have not changed; their influence on the objective has changed.

Weighted loss (conceptual)

L = Σ■ w■■ × ■■

Here, ■■ is the loss for observation i, and w■■ is the weight for its true class. A larger weight makes that observation's loss contribute more. It does not guarantee every minority example will be correctly classified.

Step 3 — What does class_weight='balanced' do?

For supported scikit-learn estimators, the common formula is w■ = N / (K × N■), where N is the training row count, K is the number of classes and N■ is the count in class k.

Class	Count	Weight calculation	Approx. weight
Legitimate (0)	990	1000 / (2 × 990)	0.505
Fraud (1)	10	1000 / (2 × 10)	50

The fraud class gets a much larger weight because it is rare. This is a frequency-based default, not a claim that every fraud error is exactly 99 times more costly in business terms.

Python example

```
from sklearn.linear_model import LogisticRegression

plain = LogisticRegression(max_iter=1000)
weighted = LogisticRegression(
    class_weight="balanced", max_iter=1000
)

plain.fit(X_train, y_train)
weighted.fit(X_train, y_train)
```

Compare both models on the same untouched test set. Check fraud precision, recall and average precision—not accuracy alone. Class weighting may improve recall, but can also increase false positives; results are data-dependent.

Memory hook: class weighting changes the **importance of errors during training**. It does not change how many rows exist.

9. Cost-sensitive learning: choose based on the cost of mistakes

Class weighting says, “make this class's training errors count more.” Cost-sensitive learning starts with a broader question: “What does each kind of mistake cost our organization?” The cost can guide training, the decision rule, or both.

Step 1 — Set illustrative business costs

Assume missing fraud costs ■5,000, while investigating a legitimate transaction incorrectly flagged as fraud costs ■200. Correct decisions have zero incremental error cost in this simplified example.

Actual class	Predict legitimate	Predict fraud
Legitimate	■0	■200 (false positive, FP)
Fraud	■5,000 (false negative, FN)	■0

Step 2 — Compare two policies

Result	Policy A	Policy B
False positives (FP)	20	60
False negatives (FN)	4	1
Cost of FP	20 × ■200 = ■4,000	60 × ■200 = ■12,000
Cost of FN	4 × ■5,000 = ■20,000	1 × ■5,000 = ■5,000
Total cost	■24,000	■17,000

Policy B creates more false alarms, but it misses fewer fraud cases. Under these assumed costs, Policy B is preferred because ■17,000 is less than ■24,000.

Step 3 — Calculate cost from predictions

Total cost = C_FP × FP + C_FN × FN

```
import numpy as np

def total_cost(y_true, y_pred,
               fp_cost=200, fn_cost=5000):
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    return fp_cost * fp + fn_cost * fn

probabilities = model.predict_proba(X_valid)[:, 1]
for threshold in [0.1, 0.2, 0.3, 0.4, 0.5]:
    pred = (probabilities >= threshold).astype(int)
    print(threshold, total_cost(y_valid, pred))
```

This evaluates different threshold policies on validation data. Select the policy using a prespecified cost objective, then evaluate it once on the untouched test set.

How are class weighting and cost-sensitive learning related?

• Class weighting is one possible way to implement cost-sensitive training.

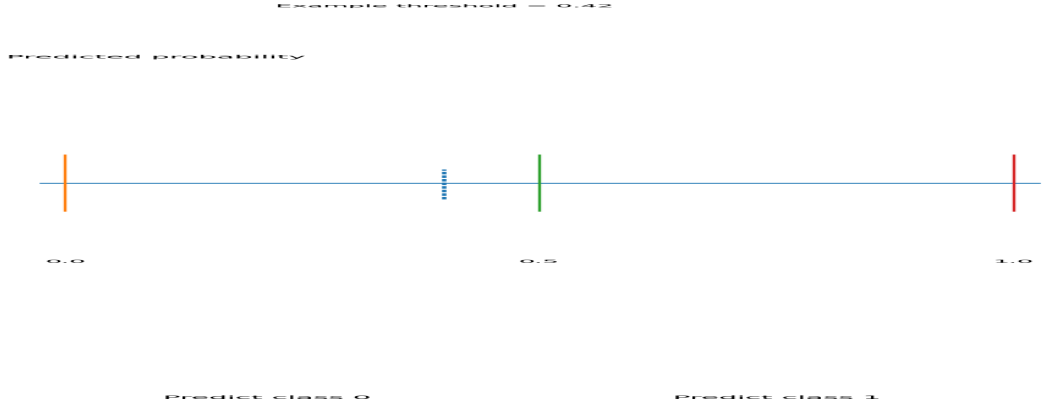
• Cost-sensitive learning is broader: it can use a weighted loss, expected-cost decision rules, or both.

• Class frequency and business cost are not the same thing. A rare class is not automatically the most expensive class.

Interview answer: optimize for the consequences of errors, not simply the count of errors. Define credible costs with domain stakeholders and validate the resulting policy.

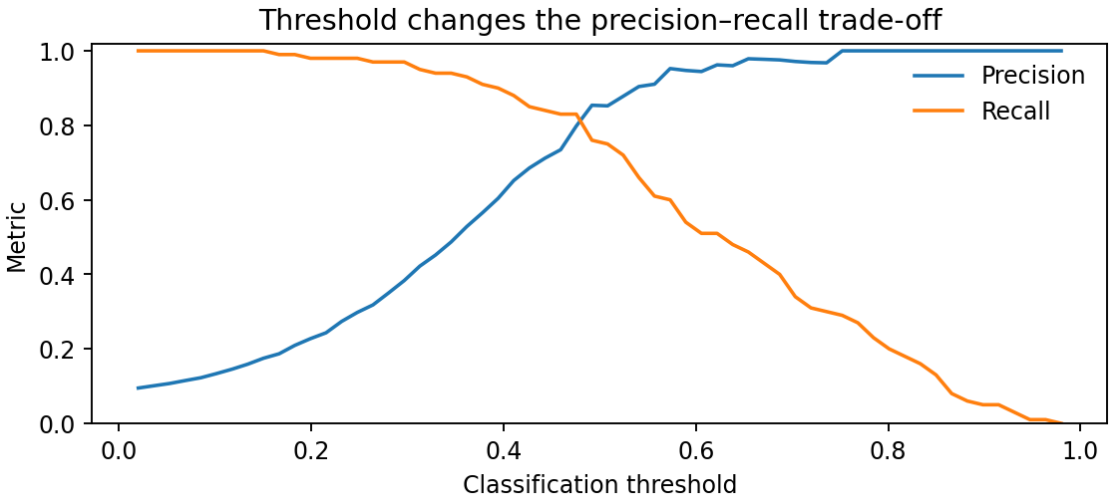
9. Threshold tuning: change the decision rule

A probabilistic classifier may estimate P(y=1 | x). A threshold converts that score into a class label. With threshold t, predict class 1 when the probability is at least t.



Why not always use 0.5?

- 0.5 is a common default, not a universal business optimum.
- Lowering the threshold usually flags more observations as positive: recall tends to rise, while precision may fall.
- Raising it usually flags fewer observations: precision may rise, while recall may fall. These are tendencies, not mathematical guarantees for every finite dataset.



The plotted curves are illustrative, generated from simulated probabilities. They show the usual trade-off, not a universal curve.

Python

```
import numpy as np
from sklearn.metrics import precision_score, recall_score, f1_score

probabilities = model.predict_proba(X_valid)[:, 1]
for threshold in [0.2, 0.3, 0.4, 0.5, 0.6]:
    pred = (probabilities >= threshold).astype(int)
    print(threshold,
          precision_score(y_valid, pred, zero_division=0),
          recall_score(y_valid, pred, zero_division=0),
          f1_score(y_valid, pred, zero_division=0))
```

Choose the threshold on validation data, using a metric or cost objective chosen in advance. Do not repeatedly tune on the final test set.

10. Which strategy should you choose?

Sampling ⟶ Changes training examples

Class weighting ⟶ Changes training loss

Ensembles ⟶ Changes learning procedure

Threshold tuning ⟶ Changes final decision rule

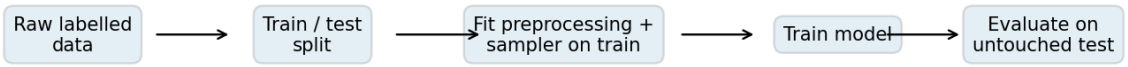
Situation	Reasonable starting point	Watch out for
Very few minority examples	Class weighting; carefully validated sampling	Synthetic samples cannot replace missing real diversity
Large redundant majority class	Random under-sampling or balanced ensemble	Discarding useful majority information
Boundary overlap / noisy neighbors	Tomek Links, ENN or hybrid cleaning	Removing valid hard examples
Need to preserve all rows	Class weighting or threshold tuning	May not solve poor feature representation
Missed positives are expensive	Recall-oriented threshold / cost-based policy	False-positive workload may rise
Need strong ranking	Compare models using PR-AUC / average precision	Ranking quality is not the same as chosen-threshold utility

A practical sequence

- 1. Establish a baseline with a stratified split and appropriate metrics.
- 2. Try class weighting and one sampling or ensemble approach.
- 3. Compare validation results at useful thresholds.
- 4. Choose based on precision/recall needs, business cost, latency and operational capacity.
- 5. Confirm performance once on the untouched test set.

There is no universally best technique. Selection is an empirical model-selection problem constrained by the real cost of errors.

11. Correct workflow: avoid data leakage



Resampling must never see validation/test observations.

Why order matters

- Split first. A sampler must never see validation or test observations.
- Fit preprocessing only on training folds. This includes scaling, imputation, feature selection and resampling.
- During cross-validation, put the sampler inside an imblearn Pipeline so each fold resamples only its own training partition.
- Keep the final test set at its natural class distribution to measure performance in the intended population.

Example: leakage-safe pipeline

```
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_validate

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("smote", SMOTE(random_state=42)),
    ("model", LogisticRegression(max_iter=1000))
])

cv = StratifiedKFold(
    n_splits=5, shuffle=True, random_state=42
)
scores = cross_validate(
    pipe, X_train, y_train, cv=cv,
    scoring=["average_precision", "recall", "precision"]
)
```

Why imblearn Pipeline?

Its sampler is applied during fit on each training fold and is not applied to the held-out fold during scoring. A standard sklearn Pipeline does not generally accept samplers that change X and y together.

Threshold tuning follows the same discipline: use validation or cross-validated out-of-fold probabilities, then evaluate the selected policy once on the test set.

12. A small comparison experiment

To understand which intervention helps, compare approaches under the same data split and evaluation setup.

Experiment	Training change	Decision change
Baseline	None	Default threshold
Weighted model	Class-weighted loss	Default threshold initially
Resampled model	Sampler inside training pipeline	Default threshold initially
Threshold-tuned model	Same fitted model	Threshold selected on validation data

Minimal comparison pattern

```
from sklearn.metrics import average_precision_score, recall_score
import numpy as np

# Fit each candidate using training data only.
# Select threshold using validation probabilities.
# Evaluate the chosen threshold on the untouched test set.

for name, fitted_model in candidates.items():
    prob = fitted_model.predict_proba(X_valid)[:, 1]
    print(name, "AP =", average_precision_score(y_valid, prob))
```

Interpretation guide

- Average precision helps compare ranking, independent of one chosen threshold.
- Recall and precision at a selected threshold describe the operating point.
- Compare false-positive volume with the team's review capacity.
- Compare expected cost when credible costs for FP and FN are available.
- Use confidence intervals or repeated cross-validation when results are noisy due to few positives.

Do not compare a baseline on the natural dataset against a model evaluated on a resampled test set. Evaluation should reflect the population where the model will be used.

13. Interview questions: explain your reasoning

Q1. Accuracy is 99%, but recall is 0%. Is the model good?

No. If positives are rare, an all-negative classifier can achieve high accuracy. Inspect the confusion matrix, recall, precision, PR-AUC and task-specific costs.

Q2. Would you always use SMOTE for a 1:99 dataset?

No. First establish a baseline. Consider class weighting, balanced ensembles and threshold tuning. SMOTE can help but can also synthesize noisy or unrealistic examples; validate rather than assume.

Q3. How are class weighting and threshold tuning different?

Class weighting changes the training loss. Threshold tuning changes how the fitted model's scores are converted to class labels. Threshold tuning can be applied without retraining, provided useful scores are available.

Q4. Why might recall rise after lowering a threshold?

More observations are labelled positive, so fewer actual positives tend to be missed (lower FN). But more negatives may also be flagged (higher FP), which can reduce precision.

Q5. When is cost-sensitive learning useful?

When error types have different consequences and those costs can be estimated. Optimize expected cost or incorporate costs into training/decisions, then validate the policy.

Q6. Why use an ensemble instead of one undersampled dataset?

Different balanced subsets can expose different majority observations to different learners. Combining learners may retain more overall majority information than training on one reduced subset, though it adds complexity.

Q7. What is the most serious sampling mistake?

Resampling before train/test splitting or before cross-validation. It can leak information and make evaluation overly optimistic.

Q8. What if the minority class has only a handful of observations?

Sampling cannot manufacture genuine diversity. Validate carefully, collect more representative examples if possible, use suitable costs/weights, and report uncertainty.

14. Five-minute revision cheat sheet

Technique	Changes	Remember
Undersampling	Training data	Less majority data; possible information loss
Oversampling / SMOTE	Training data	More minority examples; possible overfitting or synthetic noise
Hybrid sampling	Training data + cleaning	Oversample then clean; SMOTE-Tomek vs SMOTE-ENN
Balanced ensembles	Training procedure	Different balanced subsets / learners
Class weighting	Loss function	Increase the influence of minority errors
Cost-sensitive learning	Objective and/or decisions	Account for unequal error costs
Threshold tuning	Decision rule	Choose an operating point on validation data

Essential formulas

Metric	Formula
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
Specificity	TN / (TN + FP)
F1	2 × Precision × Recall / (Precision + Recall)
Balanced accuracy (binary)	(Recall + Specificity) / 2
Expected error cost	C_FP × FP + C_FN × FN

Ten rules to remember

- Accuracy alone is not enough when positive examples are rare.
- Choose metrics based on the goal and cost of errors.
- A 50:50 training set is not automatically optimal.
- Class weighting changes loss; sampling changes training examples.
- Threshold tuning changes predicted labels, not the learned model itself.
- Tomek Links finds mutual cross-class nearest neighbors; ENN checks neighborhood consistency.
- Hybrid methods combine oversampling and cleaning.
- Balanced ensembles expose learners to balanced subsets.
- Resample only within training folds; never resample the test set.
- Choose the final strategy using validation results and assess once on an untouched test set.

References for further study

• scikit-learn: Model evaluation and metrics — https://scikit-learn.org/stable/modules/model_evaluation.html
• scikit-learn: Class weights and estimator documentation — https://scikit-learn.org/stable/
• imbalanced-learn user guide — https://imbalanced-learn.org/stable/user_guide.html
• imbalanced-learn: SMOTEENN — https://imbalanced-learn.org/stable/references/generated/imblearn.combine.SMOTEENN.html
• imbalanced-learn: SMOTETomek — https://imbalanced-learn.org/stable/references/generated/imblearn.combine.SMOTETomek.html
• imbalanced-learn: BalancedRandomForestClassifier — https://imbalanced-learn.org/stable/references/generated/imblearn.ensemble.BalancedRandomForestClassifier.html
• He & Garcia (2009). Learning from Imbalanced Data. IEEE Transactions on Knowledge and Data Engineering.

This guide is a revision aid. Confirm library-version-specific parameters in the documentation for the version used in your project.